

Python ML by Example
AI Stuart Russel (Pearson)

→ AI: Mimic Human Intelligence

↳ Making machines intelligent and enable to take action & decisions that humans do

→ Machine - Makes life easier

↳ Uses power to control mechanical movements

Complex machines: 2 or more simple machines

→ Intelligence: Ability to understand, learn & think

→ 2 dimensions of AI:

(1) Humans vs. Rational

(2) Act vs. Think

→ Acting Humanly Turing Test

Total Turing Test

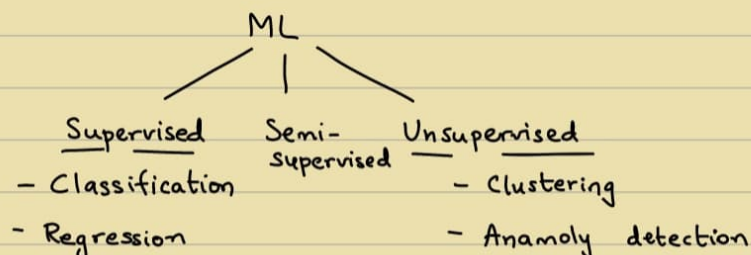
→ (1) Think Humanly

(2) Act Rationally

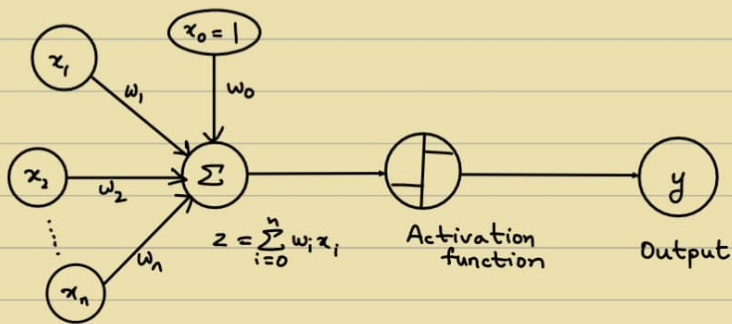
(3) Think Rationally

→ ML: Construction of Algos that can learn from & make predictions on data.

↳ features



→ Perceptron Model



$$y = f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

→ Knowledge Base :

Set of Sentences represented in knowledge representation language.

Expressing knowledge explicitly in a computer tractable way
(logic)

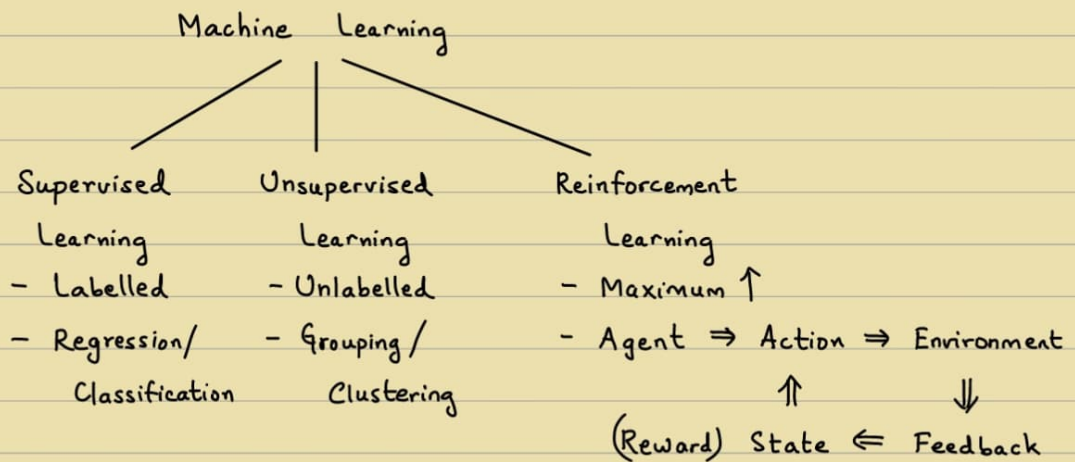
→ Logic : A way of thinking or explaining something

↳ Basis of all mathematical & Automated reasoning

17/1

→ Machine Learning :

- Perform specific tasks effectively without explicit instructions
- Relies on Patterns and Inference instead.
- Involves building a math model based on training data (sample data) to make predictions / decisions
- Why ML :
 - Data ↑ - Various formats
 - Patterns ↑



→ AlphaGo :

MuZero → Without Rules

21/1

Machine Learning

- Term coined around 1960
- Why learn ? Why not just hire enough programmers and code in rules ?
 - Lots of patterns for an activity/event
 - Events can be dynamic
 - **Data** is increasing exponentially
 - **Data** is also in various formats [Text, Audio, Video]
 - Higher quality **data** due to cheaper storage
- Can be broadly classified into three categories
 - Unsupervised, Supervised and Reinforcement learning

AlphaGo

- First computer Go program to defeat a 9-dan professional player
- Uses Monte Carlo Tree search algorithm based on knowledge learned by a deep learning method
- Beat World No. 1 ranked player in 2017
 - Retired after this match
- <https://deepmind.google/research/breakthroughs/alphago/>
- <https://www.youtube.com/watch?v=WXuK6gekU1Y>
- AlphaGo Zero – Version without human data and stronger than AlphaGo [defeated 100-0]

↪ Reinforcement learning
(Design incentive structure)

• Search:

Initially AI used BFS, then later switched to DFS.

- (1) BFS
 - (2) DFS
 - (3) A*
- } - Non-Randomised

(4) Monte Carlo Tree Search - Randomized Search Technique

Simulation - performed by Neural Networks (AlphaGo)

→ Which direction to go to ?

AlphaZero & MuZero

- AlphaZero, a generalized version of AlphaGo Zero. Took 4 hours to learn Chess and defeat reigning world computer chess champion 28 to 0 in 100 matches
- https://www.youtube.com/watch?time_continue=7&v=tXIM99xPQC8
- MuZero: Master games without knowing rules
- Uses approach similar to AlphaZero, developed in 2019

AlphaFold: solution to a 50 year old grand challenge in biology

- <https://deepmind.com/blog/article/alphafold-a-solution-to-a-50-year-old-grand-challenge-in-biology>
- Figuring out what shapes proteins fold into is known as the "protein folding problem" - grand challenge in biology for the past 50 years
- Focus of intensive scientific research for many years, using a variety of experimental techniques such as nuclear magnetic resonance and X-ray crystallography.

AlphaFold

- Number of ways a protein could theoretically fold before settling into its final 3D structure is astronomical.
- Cyrus Levinthal estimated 10^{300} possible conformations for a typical protein.
- Estimated would take longer than the age of universe to enumerate all possible configurations. Yet in nature, proteins fold spontaneously, some within milliseconds - referred to as Levinthal's paradox.

Nobel Prize in Chemistry 2024 !!!

- The Nobel Prize in Chemistry 2024 is about proteins, life's ingenious chemical tools.
- David Baker has succeeded with the almost impossible feat of building entirely new kinds of proteins.
- Demis Hassabis and John Jumper have developed an AI model to solve a 50-year-old problem: predicting proteins' complex structures. These discoveries hold enormous potential.

AI Index 2025

- Source: Stanford Institute for Human-Centered Artificial Intelligence (HAI)
- Report: 2025 AI Index Report
- <https://hai.stanford.edu/ai-index/2025-ai-index-report>
- Slides follow

AI Performance on Demanding Benchmarks

- AI achieved large gains on harder benchmarks
- Strong progress in coding and video generation
- AI agents outperform humans in time-constrained tasks

Industry Dominates AI Development

- ~90% of notable models built by industry
- Performance gaps narrowed
- Compute scale continues to rise - training compute doubles every five months, datasets every eight, and power use annually.

→ Cross - Validation :

Partitioning of training dataset.

k parts $\equiv$ Training : $k-1$ parts
 Testing : k^{th} part } → Repeat 'k' times
 k ↓

- Holdout Method - Non-exhaustive Cross Validation

Con : Some data never gets into the test data



- Exhaustive Cross-Validation :

LOOC - Used for Sparse dataset

- Nested Cross Validation :

k -fold Cross validation with validation and Test

Perform K -fold on the partitioned data (without Test)

→ Regularization :

Penalty function

Reduce importance for some terms

Occam's Razor : Simplest hypothesis that fits data should be preferred. (Not Complex model)

(i) Early Stopping

Given :

$$f(x) = \theta + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

$$MSE = \frac{1}{m} \cdot \min_{\theta, \theta_1, \theta_2, \theta_3} \sum_{i=1}^m (\hat{f}_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\text{Loss} : J(\theta)$$

Adding Penalty : $1000 \theta_3 + 1000 \theta_4$

(ii) Feature Selection

Pick top k Heuristically useful features

n features : 2^n feature combinations

Filter Methods : Correlation (Chi-Squared for Categorical)
 Pearson coeff. for Continuous

11/2

→ Feature Selection : (Reduce overfitting)

2^n total

(i) Filter Methods :

x , $2x$ - High correlation, $\therefore$ Remove Redundancy

(ii) Wrapped methods :

Underfit Model Search

Recursive feature elimination : Remove best feature and construct the next model iteratively (Worst)

(iii) Embedded methods :

Assess features acc. to Accuracy

e.g. Regularisation - Add Penalty in loss function

• Dimensionality Reduction :

- Information loss : Minimum

• Data Preprocessing :

(Data Mining)

- CRISP DM - Standard

- Data preparation $\Rightarrow$ Modeling $\Rightarrow$ Evaluation $\Rightarrow$ Deployment

- Data Cleaning - ★

Domain knowledge - ★

- Manual work (Numerical)

- Features can be quantitative / Categorical.

- Feature Engg. : Improving features

- Missing values

→ Ignore

→ Substitute with fixed values : Imputing

- Label Encoder : Label $\Rightarrow$ Numeric value

(Integer Label Encoder) - ML infers ordering based on encoding.

Done by One-hot encoding

- Scaling :

(i) Standardization : $\mu = 0$ & $\sigma^2 = 1$ $\left(\frac{z - \mu}{\sigma} \right)$

If not Gaussian, Subtract Median & Divide by Interquartile
1st (25%) and 3rd (75%) Range

(ii) Normalization : min(0) - max(1) Scaling

- Polynomial features :

Occam's Razor : Avoid Higher order & interaction terms
 $a^2 + ab + b^2$, ab : interaction term

$$y = w_1x_1 + w_2x_2 + w_3$$

- Power transformations :

- Box-Cox transform : Find best power
- Binning : Separation of feature values into bins
(Lossy transformation)

→ Uncertainty - Decision Theory

Uncertainty

- May not have categorical answers
 - May not know exactly which route is slow
 - 50% chance that one route is slow
 - Even if we know it is slow, how slow?
- **Uncertainty** changes the way an agent makes a decision
- Probability theory used in helping agents reason with uncertainty
 - Summarize uncertainty due to our ignorance or laziness
 - May be very very costly to remove such uncertainty
- **Rational decision** depends on
 - Relative importance of various goals
 - Likelihood or degree to which they will be achieved

Decisions under Uncertainty

- How should an agent make a decision:
 - Plan 1, 80% chance it will succeed (Route 1)
 - Route 1 will be quick if flowing smoothly, but stuck for 1 hour if accident
 - If plan1 is successful the outcome is very good, but failure is very bad
 - Plan 2, 70% chance it will succeed (Route 2)
 - Route 2 will be somewhat quick, but not so bad even if slow
 - If plan2 successful the outcome is good (not as good as plan1's success), but failure is not that bad

Basic Probability Notation

- Suppose we have probabilities over all possible values of a variable -- denote by $P(X)$
 - Called the probability distribution for the random variable:
 - $P(\text{Weather} = \text{sunny}) = 0.6$
 - $P(\text{Weather} = \text{rain}) = 0.1$
 - $P(\text{Weather} = \text{cloudy}) = 0.29$
 - $P(\text{Weather} = \text{snow}) = .01$
 - $P(\text{Weather}) = \langle 0.6, 0.1, 0.29, 0.01 \rangle$
- $$\sum_{xi \in \{\text{sunny, rain, cloudy, snow}\}} P(X=xi) = 1$$

Diagnosis domain: Introducing Cavity Variable

- Doctor diagnosing a dental patient's toothache
- Propositional logic rules
- Toothache $\rightarrow$ Cavity [Toothache implies Cavity] (Incomplete rule)
- Toothache $\rightarrow$ Cavity $\vee$ Gum Problem $\vee$ Abscess ... (Lots of such problems)
- Turning the rule around, Cavity $\rightarrow$ Toothache
 - Not totally right either, need to make it logically exhaustive

Inference using full joint distribution

	toothache		$\neg$ toothache	
	catch	$\neg$ catch	catch	$\neg$ catch
cavity	.108	.012	.072	.008
$\neg$ cavity	.016	.064	.144	.576

- 3 variables : {Cavity, Toothache, Catch}
- Dentist probe catches or not
- $2 * 2 * 2$ table
- $P(\text{Cavity}) = .108 + .012 + .072 + .008 = 0.2$ (Called **Marginal Probability** of Cavity)

Marginalization (or summing out)

- For the variable set Z not including variable Y

$$P(Y) = \sum_{z \in Z} P(Y, z) = \sum_z P(Y | z) P(z)$$

$$P(\text{Cavity}) = \sum_{z \in \{\text{Catch, Toothache}\}} P(\text{Cavity}, z)$$
- The above rule is called conditioning (conditioned on z)
- **Marginalization** and **conditioning** very useful rules

Marginalization

$$P(\text{cavity} | \text{toothache}) = \frac{P(\text{cavity} \wedge \text{toothache})}{P(\text{toothache})}$$

$$= \frac{0.108 + 0.012}{(0.108 + 0.102 + 0.016 + 0.064)} = 0.6$$

$$P(\neg \text{cavity} | \text{toothache}) = \frac{P(\neg \text{cavity} \wedge \text{toothache})}{P(\text{toothache})}$$

$$= \frac{0.016 + 0.064}{.108 + .012 + .016 + .064} = .4$$

$$P(\text{cavity} | \text{toothache}) + P(\neg \text{cavity} | \text{toothache}) = 1$$

Marginalization

- Denominator can be viewed as a normalization constant $1/P(\text{toothache})$
- $P(\text{Cavity} | \text{Toothache}) = \alpha P(\text{Cavity, toothache}) = \alpha [P(\text{Cavity, toothache, catch}) + P(\text{Cavity, toothache, } \neg \text{catch})] = .108 + .012 = \alpha 0.12$
- $\alpha P(\neg \text{Cavity, toothache}) = \alpha [.016 + .064] = \alpha 0.08$
- Normalizing $\alpha \langle .12, .08 \rangle$ gives $\langle .6, .4 \rangle$
- **Issue:** For a domain with n Boolean variables, input table size of $O(2^n)$
- Full joint distribution in tabular form not a practical tool for building reasoning systems

Marginalization

- Denominator can be viewed as a normalization constant $1/P(\text{toothache})$
- $P(\text{Cavity} | \text{Toothache}) = \alpha P(\text{Cavity}, \text{toothache}) = \alpha [P(\text{Cavity}, \text{toothache}, \text{catch}) + P(\text{Cavity}, \text{toothache}, \neg \text{catch})] = .108 + .012 = \alpha 0.12$
- $\alpha P(\neg \text{Cavity}, \text{toothache}) = \alpha [.016 + .064] = \alpha 0.08$
- Normalizing $\alpha <.12, .08>$ gives $<.6, .4>$
- **Issue:** For a domain with n Boolean variables, input table size of $O(2^n)$
- Full joint distribution in tabular form not a practical tool for building reasoning systems

Independence

- Helps in reducing size of domain representation and complexity of inference problem
- Let's add weather into the example : sunny, rain, cloudy, snow $\rightarrow 2 * 2 * 2 * 4$ entries in table
- $P(\text{toothache}, \text{catch}, \text{cavity}, \text{cloudy}) = P(\text{cloudy} | \text{toothache}, \text{catch}, \text{cavity}) P(\text{toothache}, \text{catch}, \text{cavity})$
- $P(\text{cloudy} | \text{toothache}, \text{catch}, \text{cavity}) = P(\text{cloudy})$

Bayes' Rule

- Given that
 - $P(A \wedge B) = P(A|B) * P(B)$
 - $P(A \wedge B) = P(B|A) * P(A)$
 - $P(B|A) = (P(A|B) * P(B)) / P(A)$
- We can determine $P(B|A)$ given $P(A|B)$, $P(B)$ and $P(A)$
- $P(\text{effect} | \text{cause})$ may be **causal knowledge**, $P(\text{cause} | \text{effect})$ **diagnostic knowledge**
- Often have conditional probabilities on causal relationships $\rightarrow P(\text{symptoms} | \text{disease})$
 - Need to infer diagnostic knowledge many times

Bayes' Rule Example

- S: Proposition that patient has stiff neck
- M: Proposition that patient has meningitis
- Meningitis causes stiff-neck, 70% of the time
- Given:
 - $P(S | M) = 0.7$
 - $P(M) = 1/50,000$
 - $P(S) = .01$
 - $P(M|S) = P(S | M) * P(M) / P(S) = (.7 * 1/50000) / .01 = .0014$

Bayes' Rule Use

- Why not estimate $P(M | S)$ the same way as we do $P(S|M)$?
 - Why do we need Bayes' rule?
- $P(S | M)$ is **causal knowledge**, does not change
 - It is "model based"
 - It reflects the way meningitis works
- $P(M | S)$ is **diagnostic**; tells us likelihood of M given symptom S
 - Diagnostic knowledge may change with circumstance, thus helpful to derive it
 - If there is an epidemic, probability of Meningitis goes up; rather than trying to estimate $P(M | S)$ which is very hard, we can compute it using the other terms
 - However, $P(S|M)$ is unaffected by the epidemic

Computing the denominator: $P(S)$

We wish to avoid computing the denominator in the Bayes' rule

- May be hard to obtain
- In our example, the denominator is $P(S)$
 - Techniques to compute (or avoid computing $P(S)$)

Our first approach is to compute relative likelihoods

- If M (meningitis) and W (whiplash) are two possible explanations:
 - $P(M|S) = P(S | M) * P(M) / P(S)$
 - $P(W|S) = P(S | W) * P(W) / P(S)$
- $P(M|S) / P(W|S) = P(S | M) * P(M) / P(S | W) * P(W)$

- Another approach: Approach #2
 - $P(M|S) = P(S | M) * P(M) / P(S)$
 - $P(\text{NOT}(M)|S) = P(S | \text{NOT}(M)) * P(\text{NOT}(M)) / P(S)$
- $P(M|S) + P(\text{NOT}(M)|S) = 1$ (These two quantities must sum to 1)
- $[P(S | M) * P(M) / P(S)] + [P(S | \text{NOT}(M)) * P(\text{NOT}(M)) / P(S)] = 1$
- $P(S | M) * P(M) + P(S | \text{NOT}(M)) * P(\text{NOT}(M)) = P(S)$
- Calculate $P(S)$ in this way...

Conditional Independence

- Toothache and Catch are independent given the presence of Cavity
 - Each is directly caused by cavity, toothache is caused by cavity and a dentist catches cavity using probe
 - A toothache cannot be caught by a probe nor a probe results in a toothache
- Conditional independence captured as : $P(\text{toothache} \wedge \text{catch} | \text{cavity}) = P(\text{toothache} | \text{catch}, \text{cavity}) P(\text{catch} | \text{cavity})$
- $P(\text{toothache} | \text{catch}, \text{cavity}) = P(A | B, C)$
 - If A and C are conditionally independent given B
 - Then, probability of A is not dependent on C
 - i.e. $P(A | B \wedge C) = P(A | B)$
- Therefore $P(\text{toothache}, \text{catch} | \text{cavity}) = P(\text{toothache} | \text{cavity}) P(\text{catch} | \text{cavity})$

More General Forms of Bayes' Rule

- $P(Y | X) = P(X | Y) * P(Y) / P(X)$
- Bayes' rule for multi-valued variables
- Generalize to some background evidence e
- $P(Y | X, e) = P(X | Y, e) * P(Y | e) / P(X | e)$

Combining Evidence

- Revisiting the Meningitis example --
- S: Proposition that patient has stiff neck
- H: Proposition that patient has severe headache
- M: Proposition that patient has meningitis
- Meningitis causes stiff-neck, 50% of the time
- Meningitis causes head-ache, 70% of the time
- Probability of Meningitis should go up, if both symptoms reported -- how to combine such symptoms?

Combining Evidence

- $P(A | B \wedge C) = P(A \wedge B \wedge C) / P(B \wedge C)$
- Numerator:
 - $P(M \wedge S \wedge H) = P(S | M \wedge H) * P(M \wedge H)$
 - $= P(S | M) * P(M \wedge H) \rightarrow \text{WHY?}$
 - $= P(S | M) * P(H | M) * P(M)$

Conditional Independence

- Helps decompose larger tables into small ones
 - Allows probabilistic systems to scale up
- Conditional independence** much more common than absolute independence
- Decomposition of large probabilistic systems into weakly connected subsets through conditional independence
 - Considered one of the most important developments in the recent history of AI
- Will deal more in lectures on Belief nets

Decision Theory

(How to make decisions)

Decision Theory

= Probability theory (deals with chance) + Utility Theory (deals with outcomes)

- Fundamental idea:**
 - The **MEU** (Maximum expected utility) principle
 - Agent is **rational** if and only if it chooses the action that yields the highest expected utility, averaged over all possible outcomes of the action
 - Weigh the utility of each outcome by the probability that it occurs

Application of MEU Principle

- $EU(\text{Plan1}) = P(\text{home-early} | \text{plan1}) * U(\text{home-early}) + P(\text{stuck1} | \text{plan1}) * U(\text{stuck1})$
 $= 0.8 * 100 + 0.2 * -1000 = -120$
- $EU(\text{Plan2}) = P(\text{home-somewhat-early} | \text{plan2}) * U(\text{home-somewhat-early}) + P(\text{stuck2} | \text{plan2}) * U(\text{stuck2})$
 $= 0.7 * 50 + 0.3 * -10 = 32$

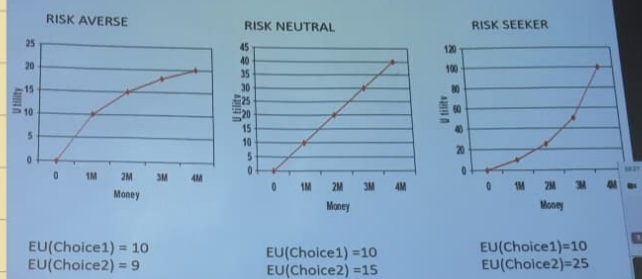
Lottery Example

- Suppose an agent gives you a choice:
 - Choice 1:** You will get \$1,000,000
 - Choice 2:** The agent will toss a coin
 - If heads, then you win \$3,000,000
 - If tails, then you get nothing
- Simple expected utility calculations give:
 - $EU(\text{Choice1}) = \$1,000,000$
 - $EU(\text{Choice2}) = \$1,500,000$
- So why did we prefer the first choice?

Risk Aversion

- We are **risk averse**
- Our utility functions for money are as follows (!!):
 - Our first million means a lot $U(\$1M) = 10$
 - Second million not so much $U(\$2M) = 15$ (NOT 20)
 - Third million even less so $U(\$3M) = 18$ (NOT 30)
 -

Risk Averse, Risk Neutral, Risk Seeking



More Risk Aversion

- Key: Slope of utility function is **continuously decreasing**
 - We will refuse to play a monetarily fair bet
- Suppose we start with x dollars
 - We are offered a game:
 - 0.5 chance to win 1000 dollars ($c = 1000$)
 - 0.5 chance to lose 1000 dollars ($c = 1000$)
 - Expected monetary gain or loss is zero (hence monetarily fair)
 - Should be neutral to it, but seems we are not! Why?
 - $U(x + c) - U(x) < U(x) - U(x - c)$
 - $U(x + c) + U(x - c) < 2 U(x)$
 - $[U(x + c) + U(x - c) / 2] < U(x)$
 - $EU(\text{playing the game}) < EU(\text{not playing the game})$

Multiattribute Utility Theory

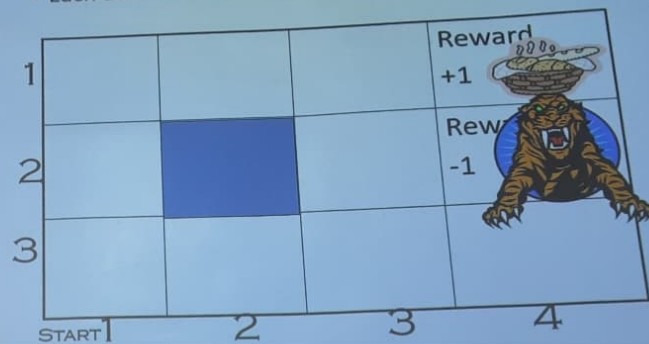
- Can we capture desirability of outcomes in a single utility function?
- Suppose renting an apartment
 - House1: closer-to-university, newer, costs 100 units
 - House2: Farther-from-university, older, costs 85 units
 - (Assume, you can afford up to 100 units)
- Outcomes characterized by two or more attributes
 - Attributes: $X_1, X_2, \dots, X_N$, e.g., <distance-to-univ, old/new, cost>
 - Values: $x_1, x_2, \dots, x_N$,
 - Closer-to-univ = 1, farther-from-univ = 0; new = 1, old = 0
 - Apartment1: $\langle 1, 1, -100 \rangle$ Apartment2: $\langle 0, 0, -85 \rangle$
 - Which is a better apartment? (Pairwise comparison fails)

Markov Decision Process (MDP) Chapter 17: Making Complex Decisions

- Defined as a tuple: $\langle S, A, P, R \rangle$
 - **S**: State
 - **A**: Action
 - **P**: Transition function
 - Table $P(s' | s, a)$, prob of s' given action "a" in state "s"
 - **R**: Reward
 - $R(s, a)$ = cost or reward of taking action a in state s

Example: What SEQUENCE of actions should our agent take?

- Agent can take action N, E, S, W
- Each action costs $-1/25$



Evaluate Markov Chain

- Use **value iteration algorithm**, but with fixed action

Initialize: $U_0(I) = 0$

Iterate:

$$U_{t+1}(I) = R(I,A) + \sum_J P(J|I,A) * U_t(J)$$

- Until close-enough (U_{t+1}, U_t)

To compare two policies:

- Check the value of start state (1,3) given the two evaluations obtained above

Value Iteration: Modify

- Initialize: $U_0(I) = 0$
- Iterate:

$$U_{t+1}(I) = \max_A [R(I,A) + \gamma \sum_J P(J|I,A) * U_t(J)]$$

– Until close-enough (U_{t+1}, U_t)

Questions

- Now that we know how to evaluate a policy, what if we just search the entire space of policies one by one? Would that give us the optimal policy?
- How do you find best policy if agent will live for 3 steps?

Policy Iteration

- Start with some (random) policy
- Repeat until policy stabilizes (no change)
 - Determine value of policy π (see previous lecture)
 - Provides us a value of each state I given policy π
 - At each state I do:
 - If /* Check if policy can be improved at each state */

$$\max_A [R(I,A) + \sum_J P(J|I,A) * U_t(J)] > R(I, \text{Policy}(I)) + \sum_J P(J|I, \text{Policy}(I)) * U_t(J)$$

Same as Value iteration formula

Linear Programming

- Mathematical programming is used to find the **best or optimal solution** to a problem that requires a decision or set of decisions about how best to use a set of limited resources to achieve a state goal of objectives.
- Steps involved in mathematical programming
 - Conversion of stated problem into a mathematical model that abstracts all the essential elements of the problem.
 - Exploration of different solutions of the problem.
 - Finding out the most suitable or optimum solution.

The Linear Programming Model (1)

Let: $X_1, X_2, X_3, \dots, X_n$ = decision variables
 Z = Objective function or linear function

Requirement: Maximization of the linear function Z .

$$Z = c_1X_1 + c_2X_2 + c_3X_3 + \dots + c_nX_n \quad \dots \text{Eq (1)}$$

subject to the following constraints:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_n$$

$$\text{all } x_j \geq 0$$

Linear Programming

- Mathematical programming is used to find the **best or optimal solution** to a problem that requires a decision or set of decisions about how best to use a set of limited resources to achieve a state goal of objectives.
- Steps involved in mathematical programming**
 - Conversion of stated problem into a mathematical model that abstracts all the essential elements of the problem.
 - Exploration of different solutions of the problem.
 - Finding out the most suitable or optimum solution.
- Linear programming** requires that all the mathematical functions in the model be **linear functions**.

The Linear Programming Model (1)

Let: $x_1, x_2, x_3, \dots, x_n$ = decision variables
 Z = Objective function or linear function

Requirement: Maximization of the linear function Z .

$$Z = c_1x_1 + c_2x_2 + c_3x_3 + \dots + c_nx_n \quad \dots \text{Eq (1)}$$

subject to the following **constraints**:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_n$$

$$\text{all } x_j \geq 0$$

The Linear Programming Model (2)

- The linear programming model can be written in more efficient notation as:

Maximize

$$Z = \sum_{j=1}^n c_j x_j$$

subject to:

$$\sum_{j=1}^n a_{ij} x_j \leq b_i$$

where

$$i = 1, 2, \dots, m$$

and

$$x_j \geq 0$$

where

$$j = 1, 2, \dots, n$$

The decision variables, $x_1, x_2, \dots, x_n$, represent levels of n competing activities.

Examples of LP Problems (1)

1. A Product Mix Problem

- A manufacturer has fixed amounts of different resources such as raw material, labor, and equipment.
- These resources can be combined to produce any one of several different products.
- The quantity of the i^{th} resource required to produce one unit of the j^{th} product is known.
- The decision maker wishes to produce the combination of products that will **maximize total income**.

Examples of LP Problems (2)

2. A Transportation Problem

- A product is to be shipped in the amounts $a_1, a_2, \dots, a_m$ from m shipping origins and received in amounts $b_1, b_2, \dots, b_n$ at each of n shipping destinations.
- The cost of shipping a unit from the i^{th} origin to the j^{th} destination is known for all combinations of origins and destinations.
- The problem is to determine the amount to be shipped from each origin to each destination such that the **total cost of transportation is a minimum**.

Examples of LP Problems (3)

3. A Flow Capacity Problem

- One or more commodities (e.g., traffic, water, information, cash, etc.) are flowing from one point to another through a network whose branches have various constraints and flow capacities.
- The direction of flow in each branch and the capacity of each branch are known.
- The problem is to determine the **maximum flow**, or capacity of the network.

Developing LP Model

XYZ Company produces two products: I and II. The raw material requirements, space needed for storage, production rates, and selling prices for these products are given in Table 1.

TABLE 1 Production Data for N. Dextrous Company

	Product	
	I	II
Storage space (ft ² /unit)	4	5
Raw material (lb/unit)	5	3
Production rate (units/hr)	60	30
Selling price (\$/unit)	13	11

The total amount of raw material available per day for both products is 1575 lb. The total storage space for all products is 1500 ft², and a maximum of 7 hours per day can be used for production.

Developing LP Model

- The object is to maximize the equation:

$$Z = 13x_1 + 11x_2$$

subject to the constraints on storage space, raw materials, and production time.

- Each unit of product I requires 4 ft² of storage space and each unit of product II requires 5 ft². Thus a total of $4x_1 + 5x_2$ ft² of storage space is needed each day. This space must be less than or equal to the available storage space, which is 1500 ft². Therefore,

$$4x_1 + 5x_2 \leq 1500$$

- Similarly, each unit of product I and II produced requires 5 and 3 lbs, respectively, of raw material. Hence a total of $5x_1 + 3x_2$ lb of raw material is used.

Developing LP Model

- This must be less than or equal to the total amount of raw material available, which is 1575 lb. Therefore,

$$5x_1 + 3x_2 \leq 1575$$
- Product I can be produced at the rate of 60 units per hour. Therefore, it must take 1/60 of an hour to produce 1 unit. Similarly, it requires 1/30 of an hour to produce 1 unit of product II. Hence a total of $x_1/60 + x_2/30$ hours is required for the daily production. This quantity must be less than or equal to the total production time available each day. Therefore,

$$x_1/60 + x_2/30 \leq 7 \text{ or } x_1 + 2x_2 \leq 420$$

Developing LP Model

- The linear programming model for this example can be summarized as:

Maximize

$$Z = 13x_1 + 11x_2$$

subject to:

$$4x_1 + 5x_2 \leq 1500$$

$$5x_1 + 3x_2 \leq 1575$$

$$x_1 + 2x_2 \leq 420$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Linear Programming for MDPs

- Maximize

$$\sum_{i \in S} V_i \quad (S \text{ is the set of states, } V_i \text{ is value of state})$$

Such that:

$$V_i \leq [R(I,A) + \gamma \sum P(J|I,A) * V_j]$$

Value Iteration Equation

Linear Programming

- Linear programming polynomial time (Karmarkar, 84)
 - Popular method Dantzig's simplex (1963)
 - Simplex performs well in practice, but could be exponential
- Although polynomial time typically slower than MDP specific algorithms such as value iteration
- Determine policy from V_i using method from previous slides
 - Yields a deterministic policy for MDPs: why?
- Key advantage: Many times it is amenable to modeling specific constraints

A more popular formulation

$$\max \sum_i \sum_a x_{ia} r_{ia}$$

$$\sum_a x_{ja} - \sum_i \sum_a x_{ia} p_{ij}^a = \alpha_j, \quad x_{ia} \geq 0$$

$x_{\{ia\}}$: Expected number of times action a is taken in state i
 $r_{\{ia\}}$: Reward for taking action a in state i
 $p_{\{a\{ij\}}}$: Probability of reaching state j when action a is taken in state i
 $\alpha_{\{j\}}$: Initial probability of being in state j

Refactoring

$$\max \sum_i \sum_a x_{ia} r_{ia} \quad \left| \quad \sum_i \sum_a (\delta_{ij} - p_{ij}^a) x_{ia} = \alpha_j, \quad x_{ia} \geq 0, \right.$$

where δ_{ij} is the Kronecker delta, defined as $\delta_{ij} = 1 \iff i = j$.

MDP Example

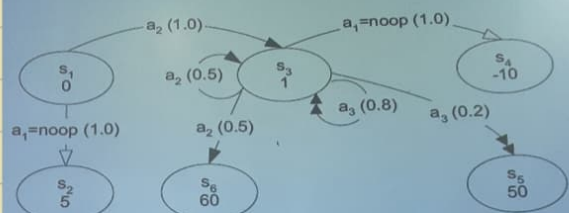
$$\max(\mathbf{r}\mathbf{x}) \mid \mathbf{A}\mathbf{x} = \alpha, \mathbf{x} \geq 0,$$

$$\mathbf{x} = [(x_{11}, x_{12}), x_{21}, (x_{31}, x_{32}, x_{33}), x_{41}, x_{51}, x_{61}]^T,$$

$$\mathbf{r} = [(0, 0), 5, (1, 1, 1), -10, 50, 60], \quad \alpha = [0.1, 0.1, 0.1, 0.1, 0.1, 0.5]^T,$$

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0.5 & 0.2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.2 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -0.5 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

MDP Example



A simple MDP: $S = \{s_1, \dots, s_6\}, A = \{a_1 = \text{noop}, a_2, a_3\}$

Solution to the LP

$$\mathbf{x} = [(0, 0.1), 0.1, (0, 0.4, 0), 0.1, 0.1, 0.7]$$

$$\mathbf{d} = [a_2, a_1, a_2, a_1, a_1, a_1]$$

Solving the LP we get $\mathbf{x}$, which maps to a deterministic uniformly-optimal policy

We get the following output if we change alpha to the following

$$\alpha = [1, 0, 0, 0, 0, 0]$$

$$\mathbf{x} = [(0, 1), 0, (0, 2, 0), 0, 0, 1]$$

$$d_1 = a_2, d_3 = a_2, d_6 = a_1$$

arbitrary actions for s_2, s_4 , and s_5

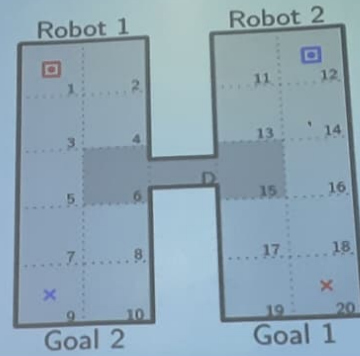
CMDP

- A key advantage of the formulation is the ability to model constraints
- If we want to say state 1 must have all actions take with equal probability
 - $x_{11} = x_{12}$
- Addition of such constraints results in a CMDP (Constrained MDP)
- How do you model action 1 in state 1 must be taken 30% of time?
 - $x_{11}/(x_{11}+x_{12}) = 1/3$
 - How can you do this with value or policy iteration?

Dec-MDP

- In MMDP model agent observes the joint state
- Many times in a joint problem, an agent may observe only his/her local state
- Separating out the policy computation for each agent is not an option since they have joint transitions and rewards
- Dec-MDP – A formal framework to address these issues

Robots coordinating in a hallway with a narrow passage



Can be modeled using Dec-MDP framework

18/4

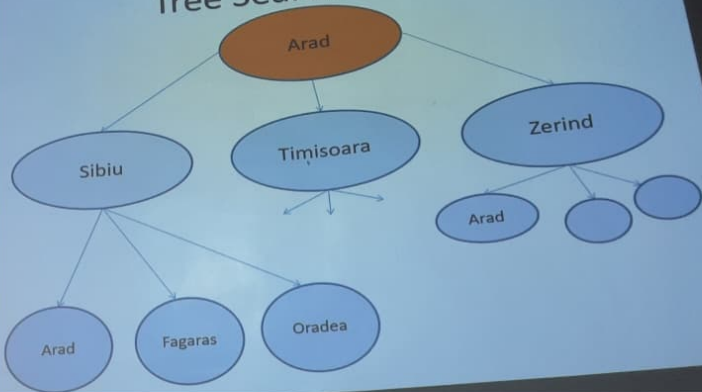
Search

- Process of looking for a **sequence of actions** that reaches the goal
- Search algorithm takes problem as input and outputs action sequence
 - Agent executes these actions
- Agent has to drive from point 1 to 2 in a map : Here environment is **observable** (agent knows current state), **discrete** (finite action set), **known** (knows states reached by actions) and **deterministic** (each action has one outcome)
 - Solution here is a fixed sequence of actions
 - Agent can ignore percepts in this case !!! (Why ???)

Searching for Solutions

- The initial state + possible action sequences form a **search tree** -- start state as root, branches as actions and nodes as states in state space
- Key steps in a searching process (Growing search tree)
 - Given a state, test if it's the goal state
 - If not, expand current state by applying set of actions
 - generates new set of states (the new leaf nodes generated called frontier)
 - Expand nodes on frontier till solution is found or no more states

Tree Search example



Graph Search vs. Tree Search

- Arad → Sibiu → Arad : Repeated Arad in search tree
- Resulted in a loopy path, a special case of redundant path (when more than 1 path exists to do same thing)
 - Ex: Arad → Sibiu (or)
 - Arad → Zerind → Ordea → Sibiu
- **Key idea:** If idea is to reach goal g that is reachable from a state s , no reason to have redundant paths to s from current state
- **Reason:** If path 1 and 2 are paths to s , extending any one of them reaches goal
- Search algorithms try to eliminate redundant paths for tractability
- Key difference between **Tree Search** and **Graph Search**

Key steps of Graph Search

- Maintain explored set (remembers expanded nodes)
- If a newly generated set already in explored set – Do not add to frontier
- Frontier separates explored vs. unexplored regions
- Hash tables for modeling explored sets – Allows efficient checking for repeated states
- Queue : FIFO – First In First Out
 - Empty(queue), Pop(queue), Insert(element, queue)
- Stack : LIFO – Last In First Out

Infrastructure for Search Algorithms

- For each node in the tree
 - **State** – Corresponds to current node
 - **Parent** – Node in search tree that generated current node
 - **Action** – Action that generated the node
 - **Path-cost** – Function of step costs that lead from initial state of node (Constructed using parent pointers)
- Function **CHILD-NODE(problem, parent, action)** returns node
 - Return a node with
 - State = problem.Result(parent.State, action)
 - Parent = parent, Action = action
 - Path-cost = parent.Path-cost + problem.Step-cost(parent.State, action)
- Terminology: node is a data structure to represent search tree vs. state is a term used in modeling the world

Measuring Performance

- **Completeness:** Is algorithm guaranteed to find solution?
- **Optimality:** Does it find the optimal solution?
- **Time Complexity:** How long it takes to find a solution?
- **Space Complexity:** How much memory to perform the search?
- Problem Size : Measured using
 - b the branching factor,
 - d the depth of shallowest goal node and
 - m the maximum length of any path in state space
- Time measured by number of nodes generated during search
- Space by maximum number of nodes stored in memory
- Total cost : Can combine search cost (typically time + maybe memory) + path cost of the solution

Uninformed Search Strategies (Chapter 3, Section 4)

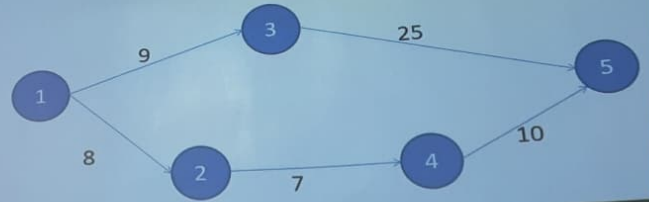
- Have no additional information apart from problem definition
- Distinguished by order in which nodes are expanded
- If we know one non-goal state is more promising than another – Called informed or heuristic search
- Breadth first search
- Uniform cost search
- Depth first search
- Depth limited search
- Iterative deepening depth first search
- Bidirectional search

Breadth first search

- **Shallowest unexpanded** node is chosen for expansion (ex: all actions have same cost)
- FIFO queue used
- Deeper nodes always at back of queue
- Goal test applied when a node is generated before adding to queue
- **Is complete:** Eventually all nodes searched
- **Is optimal:** Shallowest goal node always found first, hence optimal if path cost is non-decreasing function of depth of node
- **Time complexity:** $b + b^2 + \dots + b^d = O(b^d)$
- **Space complexity:** Graph search stores all explored nodes $O(b^d)$ (Memory is a big issue)

Uniform Cost Search

- If step costs non-uniform – Expand node n with the **lowest path cost** (instead of shallowest)
- Order queue by path cost
- Goal test applied when selected for expansion (not when generated – Why ??)



Uniform Cost Search

- **Is optimal:** If a node is selected for expansion, optimal path to it has been found
 - Otherwise it would not come up for expansion
- Assumes step costs are **non-negative** i.e. paths never get shorter
- Paths with infinite sequence of **zero-cost actions** leads to infinite loops
 - step cost of at least ϵ assumed
- Guided by path costs rather than depth

Uniform Cost Search (UCS) vs. Dijkstra Algorithm (DA)

- DA is a variant of UCS in the case where shortest path to a node needs to be found
- DA was modeled for finding shortest path to all nodes
- Hence algorithm stores all nodes upfront
 - May not be memory efficient (hence the associated run time overheads)
 - What if my goal is at depth 3 in a tree with depth 300?
- UCS is a special case of A* algorithm without a heuristic estimation part (hence slower and so is DA)

Depth First Search

- Expand **deepest** node in the frontier
- LIFO stack used
- Recursive implementations are common (Recursively expands the child)
- Graph search is complete [Tree search can have infinite loops]
 - Check new state against those on path (avoids loops)
 - Does not avoid redundant paths
- Is **non-optimal** (Returns the first encountered goal state)
- **Time complexity** for tree search $O(b^m)$ where m is the maximum depth
- **Space complexity** $O(bm)$ for tree search [for graph search the explored set is still huge]
- Depth first tree search a very popular method due to **very less memory usage**

Depth Limited Search

- Perform depth first search for a **limited depth l**
- Incomplete if $l < d$, non optimal if $l > d$
- $O(b^l)$ time and $O(bl)$ space complexity
- Depth limits can sometimes be picked based on problem
 - Ex: If 20 cities in the Romania map, longest length will be 19 from source
 - In fact, max depth is 9 steps

IDS: Iterative Deepening (Depth-First) Search

- General technique usually combined with depth first search
- Iteratively **increase the depth** searched – first 0 (root), then 1, then 2,... until a goal is found
- Combines** benefits of depth first and breadth first search
- Memory requirements less like depth first $O(bd)$
- Complete** like breadth-first and **optimal** when path cost is non-decreasing

IDS: Iterative Deepening (Depth-First) Search

- General technique usually combined with depth first search
- Iteratively **increase the depth** searched – first 0 (root), then 1, then 2,... until a goal is found
- Combines** benefits of depth first and breadth first search
- Memory requirements less like depth first $O(bd)$
- Complete** like breadth-first and **optimal** when path cost is non-decreasing
- Seems wasteful since same states generated multiple times (**but not actually**)
 - $N(\text{IDS}) = (d)b + (d-1)b^2 + \dots + (1)b^d = O(b^d)$
- Preferred search for large state spaces with unknown depth
- Iterative lengthening search – IDS version for uniform search

→ Tutorial :

Greedy Algorithm

To define any bandit algorithm formally, we first define two state variables that we track for every arm $k \in \mathcal{A}$:

- $N_k(t)$: The count of how many times arm k has been selected up to time t
- $Q_k(t)$: The estimated value (sample mean) of arm k at time t

The value of an arm is defined as the arithmetic mean of all rewards received from that arm. If arm k has been pulled n times, its estimated value is:

$$Q_k = \frac{1}{n} \sum_{i=1}^n R_i$$

All algorithms we will look at today differ only in their decision rule for picking an arm, given these two variables.

Incremental Update Rule

In practice we never store all past rewards. We can derive an extremely efficient recursive update rule that only requires us to store Q_k and N_k . Let Q_n be the estimate after n pulls. To calculate the new estimate after observing reward R_{n+1} :

$$Q_{n+1} = \frac{1}{n+1} (R_{n+1} + n \cdot Q_n)$$

$$Q_{n+1} = Q_n + \frac{1}{n+1} [R_{n+1} - Q_n]$$

This can be written in the general form that you will see again many times in reinforcement learning:

Universal Update Rule

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times [\text{Target} - \text{OldEstimate}]$$

Full Greedy Algorithm

Algorithm 1: Greedy

Initialize:

- Pull each arm $k \in \mathcal{A}$ exactly once and observe reward R_k
- Set $N_k = 1$ and $Q_k = R_k$ for all arms $k \in \mathcal{A}$

Loop for $t = K+1, K+2, \dots, T$:

- Select arm $A_t = \arg \max_k Q_k$
- Observe reward R_t
- Increment count: $N_{A_t} \leftarrow N_{A_t} + 1$
- Update estimate: $Q_{A_t} \leftarrow Q_{A_t} + \frac{1}{N_{A_t}} (R_t - Q_{A_t})$

Why The Greedy Algorithm Fails

The fundamental mathematical problem is that the sample mean estimator $\hat{\mu}_k$ only converges to the true mean μ_k if the number of samples $n \rightarrow \infty$.

In a Greedy algorithm, n only increases for the arm that already has the highest current estimate. This creates an irreversible lockout effect:

- If the optimal arm k^* yields a few low rewards early on due to random variance, its sample mean drops
- If a suboptimal arm k' yields a few lucky high rewards, its sample mean stays higher
- The algorithm will now pick k' indefinitely. Because it never picks k^* again, n_{k^*} stays constant forever, and the error is never corrected.

Why The Greedy Algorithm Fails

Let's take the simplest possible case study. Two Bernoulli arms:

- Arm 1 (Optimal): $p_1 = 0.7$
- Arm 2 (Suboptimal): $p_2 = 0.2$

Greedy algorithm initializes by pulling each arm exactly once.

$$P(\text{Arm 1 returns 0}) = 0.3$$

$$P(\text{Arm 2 returns 1}) = 0.2$$

Failure Probability

There is a $0.3 \times 0.2 = 6\%$ chance that the Greedy algorithm will get permanently and irreversibly stuck on the bad arm forever, based only on the outcome of the first two pulls.

This probability grows rapidly as you add more arms or increase reward variance.

Why The Greedy Algorithm Fails

You can frame this as a complete failure to account for uncertainty. A Greedy algorithm only ever looks at the point estimate of expected reward. It completely ignores the variance, and therefore the uncertainty, of that estimate.

The variance of a sample mean is:

$$\text{Var}(\hat{\mu}) = \frac{\sigma^2}{n}$$

If n is small, our confidence in $\hat{\mu}$ is extremely low.

The fatal flaw

The Pure Greedy algorithm treats an estimate based on 1 sample with exactly the same confidence as an estimate based on 1000 samples.

It has no concept of the value of information.

Regret

First we define the best possible performance achievable by any strategy. In a stochastic MAB, the optimal strategy is to always pull the optimal arm a^* :

$$\mu^* = \max_{k \in A} \mu_k$$

Expected Regret is the expected total reward lost by our algorithm compared to the optimal benchmark, over T steps:

$$R_T = \mathbb{E} \left[T \mu^* - \sum_{t=1}^T r_t \right]$$

The single most important identity in all of bandit theory. Using linearity of expectation we can decompose regret completely:

$$\mathbb{E}[R_T] = \sum_{k=1}^K \Delta_k \cdot \mathbb{E}[N_k(T)]$$

Where $\Delta_k = \mu^* - \mu_k$ is the gap, the fixed permanent cost of pulling arm k once.

Regret

Pseudo regret is a cleaner version used almost exclusively in all modern proofs. Almost everyone confuses these two definitions.

While Expected Regret compares the optimal mean to the actual noisy rewards we received, Pseudo-regret $\bar{R}_T$ compares the optimal mean to the true mean of the arm we chose:

$$\bar{R}_T = T \mu^* - \sum_{t=1}^T \mu_{a_t}$$

Key Difference

Pseudo regret counts how much reward you should have lost on average, before you see the actual random reward. It completely removes the noise of individual rewards from the analysis.

Why Linear Regret Is A Complete Failure

We formally define an algorithm as no-regret if its average regret per step vanishes to zero as time goes to infinity:

$$\lim_{T \rightarrow \infty} \frac{\mathbb{E}[R_T]}{T} = 0$$

This is the mathematical definition of an algorithm that actually learns.

Regret Order	Asymptotic Behaviour
$O(T)$	$\lim_{T \rightarrow \infty} \frac{R_T}{T} = C$, a positive constant. The algorithm never stops making mistakes at a fixed rate.
$O(\log T)$	$\lim_{T \rightarrow \infty} \frac{R_T}{T} \rightarrow 0$ The algorithm converges to the optimal arm.

Why Greedy Has Linear Regret

We can now write the exact asymptotic regret of Pure Greedy explicitly.

Recall that Pure Greedy has a constant probability δ of getting permanently and irreversibly locked out of the optimal arm forever.

If it gets locked out, it will pull the suboptimal arm for every single remaining step, incurring the full gap Δ every time.

Therefore the total expected regret of Pure Greedy is:

$$\mathbb{E}[R_T] = \delta \cdot \Delta \cdot T$$

Since δ and Δ are constants, completely independent of T , the regret of Pure Greedy is exactly linear $O(T)$.
Pure Greedy is not a no-regret algorithm.

The Great Epsilon Greedy Lie

This is the most important surprise in this tutorial. Almost everyone thinks epsilon greedy fixes all the problems of pure greedy. It does not. It fixes the permanent lockout problem. But it has exactly the same asymptotic regret order as pure greedy.

The ϵ -Greedy policy selects a random arm with a fixed probability ϵ at every single time step t , forever.

Let A_{sub} be the set of all suboptimal arms. The probability of picking a suboptimal arm at any time t is always at least:

$$P(A_t \in A_{sub}) \geq \epsilon \cdot \frac{K-1}{K}$$

Because ϵ is a constant that does not depend on t , the agent will never stop sampling suboptimal arms, even as $t \rightarrow \infty$.

Linear Regret of Epsilon Greedy

Recall the definition of pseudo regret:

$$\bar{R}_T = \sum_{t=1}^T (\mu^* - \mu_{a_t})$$

Suppose after some time the algorithm has correctly identified the optimal arm, so $\arg \max Q_k = a^*$. The expected loss at every single future timestep is:

$$\mathbb{E}[\text{loss}_t] = (1 - \epsilon) \cdot 0 + \epsilon \cdot \text{Avg}(\Delta)$$

Where $\text{Avg}(\Delta)$ is the average gap of all suboptimal arms. Since ϵ and the gaps are all constants:

$$\mathbb{E}[\text{loss}_t] = C \quad \text{where } C > 0$$

Dynamic ϵ -Greedy: A Natural Fix

The root cause of linear regret in fixed ϵ -Greedy is clear: we keep exploring at a constant rate ϵ even after we have confidently identified the optimal arm.

Intuition: We should explore a lot early on, when our estimates are uncertain, and explore less and less as our estimates improve.

The simplest way to implement this: let ϵ decay with time.

Algorithm 3: Dynamic ϵ_t -Greedy

Replace the fixed ϵ with a schedule ϵ_t that decreases over time. A natural choice:

$$\epsilon_t = \frac{1}{t}$$

Everything else is identical to ϵ -Greedy. At time t , explore with probability $\frac{1}{t}$ and exploit otherwise.

Regret of Dynamic $\epsilon_t = 1/t$ Greedy

Since $\epsilon_t \rightarrow 0$, it appears we explore less and less. But does this achieve sub-linear regret?

Recall the expected loss at step t (assuming the optimal arm is identified):

$$\mathbb{E}[\text{loss}_t] = \epsilon_t \cdot \text{Avg}(\Delta) = \frac{C}{t}$$

Summing over all T steps:

$$\mathbb{E}[R_T] \leq C \sum_{t=1}^T \frac{1}{t} = C \cdot H_T$$

where $H_T = \sum_{t=1}^T \frac{1}{t}$ is the T -th harmonic number.

Recall from Analysis

The harmonic series diverges logarithmically:

$$H_T = \sum_{t=1}^T \frac{1}{t} = \ln T + O(1)$$

Dynamic ϵ -Greedy is $O(\log T)$... But Still Suboptimal

Putting it together:

$$\mathbb{E}[R_T] = O(\log T)$$

Great news and bad news

Good: This is sub-linear. The algorithm genuinely learns. Average regret per step $\rightarrow 0$.

Bad: It is *not* the best possible $O(\log T)$. The constant hidden inside the $O(\cdot)$ is much larger than it needs to be.

Why is it suboptimal? The $1/t$ schedule is *oblivious* to the actual problem structure. It does not use the gaps Δ_k . It over-explores easy problems (large gaps) and under-explores hard problems (small gaps).

A principled algorithm should allocate exploration *precisely* according to how uncertain each arm estimate is, and how much that uncertainty matters.

What is the Optimal Regret?

A natural question: is $O(\log T)$ optimal, or can we do better?

Lai & Robbins (1985) Lower Bound

For any consistent algorithm (one that achieves sub-linear regret on every bandit instance), the expected number of times a suboptimal arm k is pulled must satisfy:

$$\liminf_{T \rightarrow \infty} \frac{\mathbb{E}[N_k(T)]}{\ln T} \geq \frac{1}{\text{KL}(\mu_k \parallel \mu^*)}$$

where $\text{KL}(\mu_k \parallel \mu^*)$ is the KL-divergence between the two arm distributions.

This gives an instance-dependent lower bound on total regret:

$$\mathbb{E}[R_T] \geq \left(\sum_{k: \Delta_k > 0} \frac{\Delta_k}{\text{KL}(\mu_k \parallel \mu^*)} \right) \ln T$$

Key Takeaway

$\Omega(\log T)$ regret is *unavoidable*. No algorithm can do better in the worst case. The goal is to match this bound, up to constant factors.

UCB1 Algorithm

Choose $\alpha = t^{-4}$ (we explain this choice in the proof). Then the confidence bonus for arm k at time t is:

$$U_k(t) = \sqrt{\frac{2 \ln t}{N_k(t)}}$$

Algorithm 4: UCB1 (Auer et al., 2002)

Initialize: Pull each arm exactly once (steps 1 to K).

Loop for $t = K + 1, K + 2, \dots, T$:

1. Select arm: $A_t = \arg \max_{k \in \mathcal{A}} \left[Q_k(t) + \sqrt{\frac{2 \ln t}{N_k(t)}} \right]$
2. Observe reward R_t
3. Update: $N_{A_t} \leftarrow N_{A_t} + 1$, $Q_{A_t} \leftarrow Q_{A_t} + \frac{1}{N_{A_t}}(R_t - Q_{A_t})$

Notice: No ϵ hyperparameter. The algorithm is entirely self-tuning. The bonus automatically shrinks as $N_k(t)$ grows, and grows as $\ln t$ slowly increases.

Upper Confidence Bound (UCB): Intuition

The core insight missing from both Greedy and ϵ -Greedy:

Exploration should be directed at arms whose true value is *uncertain*, not chosen uniformly at random.

Optimism in the face of uncertainty.

Rather than using the point estimate Q_k , UCB augments it with an *upper confidence bonus*:

$$\text{UCB}_k(t) = \underbrace{Q_k(t)}_{\text{exploitation}} + \underbrace{U_k(t)}_{\text{exploration bonus}}$$

The bonus $U_k(t)$ is designed so that with high probability:

$$\mu_k \leq Q_k(t) + U_k(t)$$

We are optimistic: we pick the arm that *could plausibly* be the best.

Key Tool: Hoeffding's Inequality

To derive the confidence bonus, we need a concentration inequality.

Hoeffding's Inequality

Let $X_1, \dots, X_n$ be i.i.d. random variables bounded in $[0, 1]$. Let $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i$. Then for any $\delta > 0$:

$$P(\hat{\mu} - \mu \geq \delta) \leq e^{-2n\delta^2}$$

and symmetrically:

$$P(\mu \geq \hat{\mu} + \delta) \leq e^{-2n\delta^2}$$

Setting the right-hand side to some small probability α and solving for δ :

$$e^{-2n\delta^2} = \alpha \implies \delta = \sqrt{\frac{\ln(1/\alpha)}{2n}}$$

This is the half-width of our confidence interval. With probability $1 - \alpha$, the true mean lies within δ of our estimate.

UCB1 Regret Theorem

Theorem (UCB1 Regret Bound)

For a K -arm bandit with rewards in $[0, 1]$, the UCB1 algorithm satisfies:

$$\mathbb{E}[R_T] \leq \sum_{k: \Delta_k > 0} \left(\frac{8 \ln T}{\Delta_k} + \left(1 + \frac{\pi^2}{3}\right) \Delta_k \right)$$

Reading this bound:

- The dominant $O(\log T)$ term is *inversely proportional* to Δ_k : arms that are hard to distinguish from the optimal arm get pulled more often.
- The constant $(1 + \pi^2/3) \approx 4.3$ is a small per-arm additive overhead.
- This matches the Lai-Robbins lower bound up to constant factors for bounded rewards.

Assembling the UCB1 Regret Bound

Recall the regret decomposition:

$$\mathbb{E}[R_T] = \sum_{k: \Delta_k > 0} \Delta_k \cdot \mathbb{E}[N_k(T)]$$

Substituting our bound $\mathbb{E}[N_k(T)] \leq \frac{8 \ln T}{\Delta_k^2} + 1 + \frac{\pi^2}{3}$:

$$\begin{aligned} \mathbb{E}[R_T] &\leq \sum_{k: \Delta_k > 0} \Delta_k \left(\frac{8 \ln T}{\Delta_k^2} + 1 + \frac{\pi^2}{3} \right) \\ &= \sum_{k: \Delta_k > 0} \left(\frac{8 \ln T}{\Delta_k} + \left(1 + \frac{\pi^2}{3}\right) \Delta_k \right) \end{aligned}$$

UCB1 Regret (Final)

$$\mathbb{E}[R_T] \leq \sum_{k: \Delta_k > 0} \frac{8 \ln T}{\Delta_k} + \left(1 + \frac{\pi^2}{3}\right) \sum_{k: \Delta_k > 0} \Delta_k = O(\log T) \quad \square$$

UCB1 achieves $O(\log T)$ regret with instance-dependent constants that depend on the gaps Δ_k . This matches the Lai-Robbins lower bound up to constants, making UCB1 an asymptotically optimal algorithm.

Use Sir's Notation and Be consistent with the notations

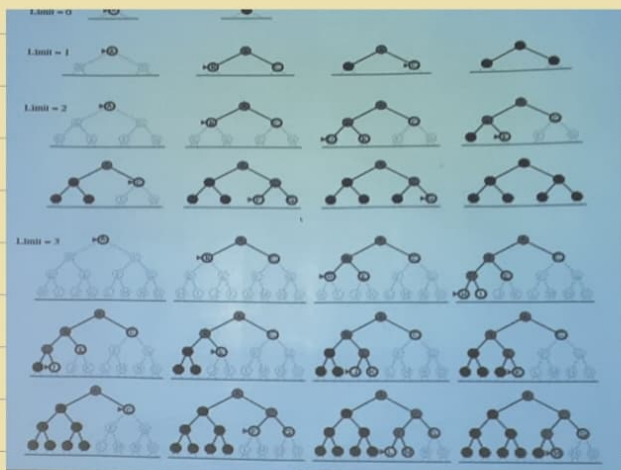
Definitions

Dry Run of Algos

IDS: Iterative Deepening (Depth-First) Search

- General technique usually combined with depth first search
- Iteratively **increase the depth** searched – first 0 (root), then 1, then 2,.... until a goal is found
- **Combines** benefits of depth first and breadth first search

• Depth - Limited Search :



IDS: Iterative Deepening (Depth-First) Search

- General technique usually combined with depth first search
- Iteratively **increase the depth** searched – first 0 (root), then 1, then 2,.... until a goal is found
- **Combines** benefits of depth first and breadth first search
- Memory requirements less like depth first $O(bd)$
- **Complete** like breadth-first and **optimal** when path cost is non-decreasing
- Seems wasteful since same states generated multiple times (**but not actually**)
 - $N(\text{IDS}) = (d)b + (d-1)b^2 + \dots + (1)b^d = O(b^d)$
- Preferred search for large state spaces with unknown depth
- Iterative lengthening search – IDS version for uniform search

Bidirectional Search

- A **forward and backward** breadth first search simultaneously
- $b^{(d/2)} + b^{(d/2)}$ is much less than b^d
- $O(b^{d/2})$ time and space complexity
- A solution will be present at the intersection of frontiers (additional checks needed for optimality)
- How to search backwards ?
 - Need to compute predecessors
- If multiple goals, dummy goal state whose immediate predecessors are the goal states
- Search difficult if goal state is abstract ex: no queen attacks another queen

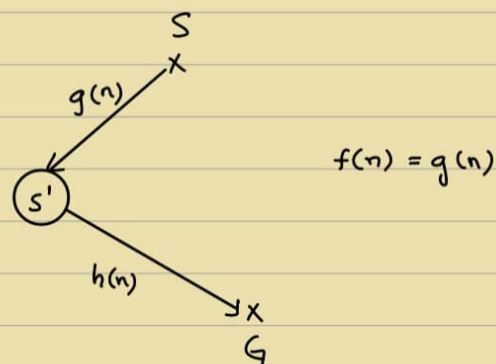
→ Informed Search :

Best First Search (Chapter 3, Section 5)

- Node selected for expansion based on evaluation function $f(n)$
- Node with lowest cost evaluation expanded first
- Heuristic function $h(n)$ as a component of $f(n)$
- $h(n)$ = *estimated cost of the cheapest path from the state at node n to a goal state*
- **Greedy Best First Search**
- **A* Search**
- **Memory bounded heuristic search**
 - IDA*, RBFS and SMA*

Greedy Best First Search

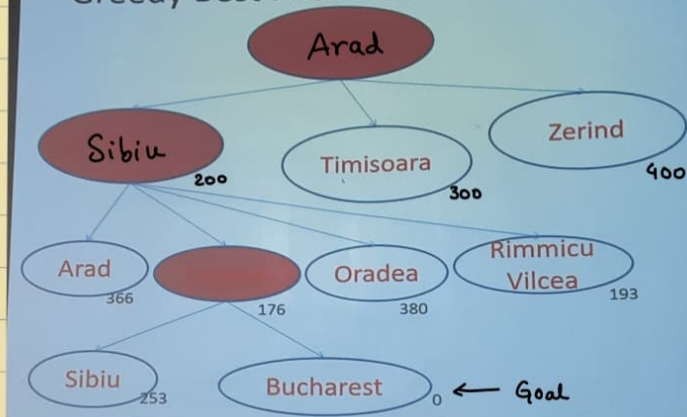
- Expand node closest to goal
- $f(n) = h(n)$
- For route finding problem straight line distance h_{SLD} can be a heuristic



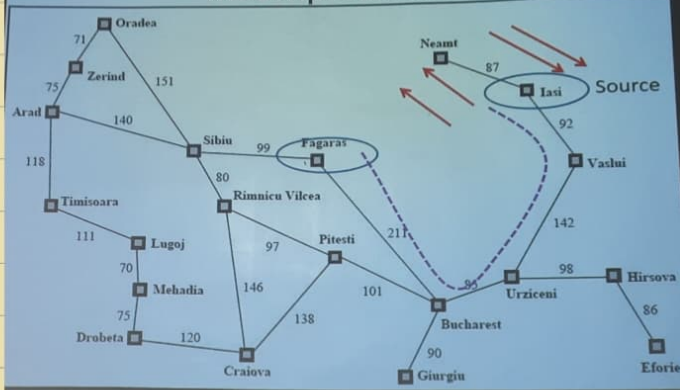
Greedy Best First Search

- Expand node closest to goal
- $f(n) = h(n)$
- For route finding problem straight line distance h_SLD can be a heuristic
- h_SLD from Arad to Bucharest is 366 (Next slide)
- Need not** be optimal
- Tree search is **incomplete** like depth first (infinite loop), graph search is **complete** for finite spaces

Greedy Best First Tree Search Example



Incompleteness



A* Search

- $f(n) = g(n) + h(n)$ i.e. Cost to reach the node + Cost from node to goal
- $f(n)$ = Estimated cost of the cheapest solution through n

Conditions for optimality: Admissibility and Consistency

- Admissibility:** $h(n)$ never overestimates the cost to reach a goal
- Optimistic in nature since $h(n)$ thinks cost of solving problem is lower than it actually is
- Straight line distance h_LSD is admissible
 - Cannot have something shorter than straight line
- Consistency (or monotonicity):** Required for graph search
- $h(n) \leq c(n, a, n') + h(n')$
- Form of triangle inequality – Triangle formed by n , n' and goal G_n
- If there is a route from n to G_n via n' cheaper than $h(n)$ it would violate $h(n)$ as lower bound to reach G_n

Optimality of A*

- Every** consistent heuristic is admissible
- Consistency is a **stricter** requirement although most heuristics are both admissible and consistent
- Is h_SLD a consistent heuristic? ← H.W
- Tree search version optimal if $h(n)$ is admissible and graph search optimal if $h(n)$ is consistent
- If A* selects a path p , then p is the shortest or lowest cost path

Proof of optimality for A*

- Step 1:** If $h(n)$ is consistent, then values of $f(n)$ along any path are non decreasing
- Step 2:** Whenever A* selects a node n for expansion, the optimal path to that node has been found
- For Step 1:** For definition of consistency, if n' is successor of n
 - $g(n') = g(n) + c(n, a, n')$
 - $f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n) \Rightarrow f(n') \geq f(n)$

Proof of optimality for A*

- For Step 2:** If optimal path to node n was not found before expanding
 - Another frontier node n' would be on the optimal path from start node to n
 - By the graph separation property (frontier separates the explored region of the state space from the unexplored region), since f is non-decreasing n' will have lower cost than n and hence would be selected first
- From steps 1 + 2, sequence of nodes expanded is in **non decreasing** order of $f(n)$
- Hence the **first goal** node selected must be optimal and later goal nodes will be at least as expensive

A* properties

- A* expands all nodes with $f(n) < C^*$
- A* might expand nodes on goal contour ($f(n) = C^*$) before selecting a goal node
- A* does not expand nodes with $f(n) > C^*$
- A* is complete i.e. will always find a solution
- A* is optimally efficient for any given consistent heuristic
 - No other optimal algorithm is guaranteed to expand fewer nodes than A* given the heuristic
 - If an algorithm does not expand all nodes with $f(n) < C^*$, it may miss the optimal solution
- Hence A* is **complete, optimal and optimally efficient**

A* properties

- Complexity of A* dependent on quality of heuristic function chosen, varies with assumptions made on state space
- **Absolute error** $\Delta \equiv h^* - h$ (h^* is actual cost of getting from root to goal, h the estimated cost)
- **Relative error** $\epsilon \equiv (h^* - h)/h^*$
- For tree with reversible actions and single goal, $O(b^{\Delta})$
 - With constant step costs, $O(b^{\Delta}) = O((b^{\epsilon})^d)$
- With multiple goal states
 - Extra cost proportional to number of goals (whose cost is within factor ϵ of optimal cost)
- For general graph, exponentially many states with $f(n) < C^*$
- While computationally expensive to find optimal solutions, usually runs out of space before time